N. Robertson and P. D. Seymour, Graph Minors. II. Algorithmic Aspects of Tree-Width, *J. Algorithms* **7** (1986), 309–322.

About this copy

This is a corrected copy of the paper cited above. The text is retyped from the printed paper. It keeps the paper's wording and the numbering of its results, equations and footnotes, and a marker such as [p. N] shows where page N of the printed paper begins.

The corrections come from a formalization of the Graph Minors series in the Lean proof assistant, github.com/mahdi-jfri/GraphMinor, in which the results of this paper are proved. Where the formalization found that a statement or a step of a proof is wrong as printed, this copy corrects it in place. Where the paper leaves a proof or a step to the reader, this copy adds it, also in blue. Text ~~struck through in red~~ is removed and text in blue is inserted. Each change has a note, marked F1, F2, ..., that gives the reason. The file corrections/GM02.md of the formalization describes each correction in full. Each note, and each item in the list below, begins with the kind of the change. A **misprint** is a wrong symbol, index or word. A **statement mistake** is a result or a claim of the text that is false as printed. A **degenerate case** is a statement that fails only in a trivial case, such as an empty graph or a parameter equal to 0. A **proof gap** is a step of a proof that does not follow as written. A **missing proof** is a proof that the paper omits, leaves to the reader or only sketches, and that this copy adds.

The corrections and added proofs, in the order of the pages:

1. **Misprint.** Statement of (2.3), page 311: the second v in "$v, v \notin X_i$" becomes v'.
2. **Misprint.** Proof of (2.5), page 313: $|W' \cup Q|$ becomes $|W' \cap Q|$.
3. **Missing proof.** Proof of (2.7), page 314: the proof is added.
4. **Missing proof.** Proof of (2.8), page 315: the proof of the bound is added.
5. **Statement mistake.** Section 2, page 315: the induced terminated graphs also lose the terminals of $\mathscr{A}_0$ in X that their own termination does not use.
6. **Missing proof.** Proof of (2.9), page 315: the proof is added.
7. **Statement mistake.** Statement of (4.5), page 319: the hypothesis that H has no loops and no two edges with the same ends is added.

8. **Proof gap.** Proof of (4.6), page 319: the construction of G' is changed so that it remembers the edges of G.
9. **Missing proof.** Proof of (4.8), page 320: the details of the splitting argument are added.
10. **Missing proof.** Proof of (5.4), page 321: the proof is added.

[p. 309] Graph Minors. II. Algorithmic Aspects of Tree-Width

NEIL ROBERTSON AND P. D. SEYMOUR*
Ohio State University, Department of Mathematics, 231 West 18th Avenue,
Columbus, Ohio 43210
Received January 15, 1983

We introduce an invariant of graphs called the *tree-width*, and use it to obtain a polynomially bounded algorithm to test if a graph has a subgraph contractible to H, where H is any fixed planar graph. We also nonconstructively prove the existence of a polynomial algorithm to test if a graph has tree-width $\leq w$, for fixed w. Neither of these is a practical algorithm, as the exponents of the polynomials are large. Both algorithms are derived from a polynomial algorithm for the DISJOINT CONNECTING PATHS problem (with the number of paths fixed), for graphs of bounded tree-width.

1. INTRODUCTION

All graphs in this paper are finite, and may have loops or multiple edges. $V(G)$ and $E(G)$ denote the sets of vertices and edges of the graph G, respectively. Let G be a graph. A *tree-decomposition* of G is a family $(X_i : i \in I)$ of subsets of $V(G)$, together with a tree T with $V(T) = I$, with the following properties.

(W1) $\bigcup(X_i : i \in I) = V(G)$.

(W2) *Every edge of G has both its ends in some X_i $(i \in I)$.*

(W3) *For $i, j, k \in I$, if j lies on the path of T from i to k then* $X_i \cap X_k \subseteq X_j$.

The *width* of the tree-decomposition is $\max(|X_i| - 1 : i \in I)$. The *tree-width* of G is the minimum $w \geq 0$ such that G has a tree-decomposition of width $\leq w$. (Equivalently, the tree-width of G is the minimum $w \geq 0$ such [p. 310] that G is a subgraph of a "chordal" graph with all cliques of size at most $w + 1$.)

*Research partially supported by NSF Grant MCS 8103440. Present address: Bell Communications Research, Inc., Morris Research and Engineering Center, 435 South Street, Morristown, New Jersey 07960.

Then, for example, trees and forests have tree-width 1 (or 0 in degenerate cases) and it can be shown that series-parallel graphs have tree-width ≤ 2. For $n \geq 1$, the complete graph K_n has tree-width $n-1$, while the $n \times n$ grid (this has n^2 vertices—it is the adjacency graph of the chessboard with n^2 squares), for $n \geq 2$, has tree-width n.

A graph G is *contractible* to H when H can be obtained from G by contracting edges. H is a *minor* of G if G has a subgraph contractible to H. Other papers in this series [9, 10, 12, 14] establish a connection between the absence of a fixed graph H as a minor of an arbitrary graph G and the presence of a fixed type of structure on G. For example, in [12] it is shown that *if H is a planar graph there is a number w, depending only on H, such that any graph G with no minor isomorphic to H has tree-width $\leq w$.* Such structure can be exploited to obtain, by developments of standard methods, results which have defied solution for graphs in general. Thus, in [11] it is shown that *any set of graphs with bounded tree-width, no member of which is isomorphic to a minor of another member, must be finite.* This was proved by Kruskal [5] for graphs of tree-width ≤ 1, under the stronger inclusion relation of topological containment.

The main object of this paper is to use the result from [12] to obtain an algorithm for testing minor inclusion of a fixed planar graph H. The *size* of a graph G is $|V(G)| + |E(G)|$. If the running time of a graph algorithm can be bounded above by a polynomial in the size of the input graph, it is called a *polynomial algorithm.* Our main results are as follows.

(1.1) *For any fixed planar graph H, there is a polynomial algorithm to decide if the input graph has a minor isomorphic to H.*

(1.2) *For any fixed integer w, there is a polynomial algorithm to decide if the input graph has tree-width $\leq w$.*

(1.3) *For any fixed integers k, w there is a polynomial algorithm to solve the* DISJOINT CONNECTING PATHS *problem (stated below) for k paths, in graphs of tree-width $\leq w$.*

Our proofs for (1.1) and (1.3) are constructive (we find the algorithms), but for (1.2) the proof is nonconstructive. We should stress that these make no claim to be practical algorithms, for the exponent of the polynomial is very large, even for quite small H, k, w. Our concern is more theoretical, with the *NP*-completeness (or lack of it) of problems of this type. (We shall assume familiarity with this concept—see [2].)

A *termination* $\mathscr{A}$ in a graph G is a set $\mathscr{A} = \{(s_i, t_i) : 1 \leq i \leq k\}$ of pairs of vertices of G, such that $s_1, t_1, s_2, t_2, \ldots, s_k, t_k$ are all distinct. The *support* $S(\mathscr{A})$ of $\mathscr{A}$ is $\{s_1, t_1, \ldots, s_k, t_k\}$. The pair $(G, \mathscr{A})$ is a *terminated*

graph and it has *index* k. The terminated graph is *feasible* if there are paths [p. 311] $P_1, \ldots, P_k$ of G, pairwise vertex-disjoint, such that P_i joins s_i and t_i $(1 \le i \le k)$.

The problem DISJOINT CONNECTING PATHS is: *Given a terminated graph, decide if it is feasible.* It is known [3] that DISJOINT CONNECTING PATHS is *NP*-complete. However, this depends on the index being a variable part of the input. The problem is not known to be *NP*-complete for any fixed index. Indeed, we conjecture that for fixed index there is a polynomial algorithm to solve it. This is so for index 2 [15, 16, 17] and for arbitrary fixed index if the graph is restricted to lie on a fixed surface [13]. (Incidentally, even for planar graphs the problem is *NP*-complete if we do not bound the index [6, 8].)

The key result of the present paper (essentially (1.3)) is that for any fixed numbers w, k there is a polynomial algorithm which, given as input a terminated graph with index $\le k$, either decides if it is feasible or outputs that the graph has tree-width $> w$. The next section contains some preliminaries to the proof of this, and the proof itself is given in Section 3. In Sections 4 and 5 we derive (1.1) and (1.2), using (1.3) and the results of [11] and [12] stated above.

2. Preliminaries

Suppose we have a tree-decomposition of G, with notation as before. For each vertex $i \in I$ of the tree T, the connected components of $T \setminus i$ are called the *branches of T at i*. [$G \setminus X$ denotes the graph obtained by deleting X from G, where X may be a vertex or edge, or a set of vertices or edges.] The branches at i are in a natural correspondence with the edges of T incident with i.

(2.1) *For $i \in I$ and $v \in V(G)$, either $v \in X_i$, or there is a branch of T at i which contains all $j \in I$ with $v \in X_j$.*

This follows immediately from (W1), (W3). If $v \notin X_i$, let $T_i(v)$ denote this branch. (It is unique).

(2.2) *If $v, v' \notin X_i$ and v, v' are adjacent in G then $T_i(v) = T_i(v')$.*

Proof. By (W2), there exists $j \in I$ such that $v, v' \in X_j$. Then $j \ne i$, and so j is in some branch B of T at i. But then $B = T_i(v)$, because $v \in X_j$, and similarly $B = T_i(v')$.

(2.3) *If $v, \not{v}v' \notin X_i$[F1] and are not separated in G by X_i, then $T_i(v) =$*

[F1] **Misprint.** The conclusion is about two vertices v and v', and (2.2) has "$v, v' \notin X_i$". As printed, nothing is assumed about v'.

$T_i(v')$.

[Y, Y' are *separated* by X in G if every path in G from Y to Y' meets X. Here X, Y, Y' are vertices or sets of vertices.]

[p. 312] *Proof.* Let $v_1, v_2, \ldots, v_r$ be a sequence of vertices of G not in X_i, each adjacent to the next, with $v_1 = v$ and $v_r = v'$. Then by (2.2),

$$T_i(v_1) = T_i(v_2) = \cdots = T_i(v_r)$$

and so $T_i(v) = T_i(v')$.

(2.4) *Let e be an edge of T with ends j, j' say, and N, N' be the vertex sets of the two components of $T \setminus e$. Then $X_j \cap X_{j'}$ separates $\bigcup(X_i : i \in N)$ and $\bigcup(X_i : i \in N')$.*

Proof. Suppose not. Then from (W1), there exist

$$v, v' \in V(G) - (X_j \cap X_{j'})$$

with $v \in X_i$ and $v' \in X_{i'}$ say, where $i \in N$ and $i' \in N'$, such that either $v = v'$ or v, v' are adjacent in G. From (W1), (W2) there exists $k \in I$ such that $v, v' \in X_k$. We assume without loss of generality that $k \in N'$. Then j, j' are both on the path of T between i and k, and so $X_i \cap X_k \subseteq X_j, X_{j'}$ by (W3). Hence $v \in X_j \cap X_{j'}$, a contradiction, as required.

(2.5) *If the tree-decomposition has width $< w$ and $Q \subseteq V(G)$, then there exists $X \subseteq V(G)$ with $|X| \leq w$ such that every component of $G \setminus X$ has at most $\frac{1}{2}|Q - X|$ vertices which are in Q.*

Proof. There are two cases.

Case 1. *Some $i \in I$ has the property that for each branch B of T at i,*

$$|\{q \in Q - X_i : T_i(q) = B\}| \leq \tfrac{1}{2}|Q - X_i|.$$

Then we may take $X = X_i$. For $|X_i| \leq w$, and if C is a component of $G \setminus X_i$, all vertices v of C have $T_i(v)$ the same, by (2.3), and so C has at most $\frac{1}{2}|Q - X_i|$ vertices which are in Q, as required.

Case 2. *For each $i \in I$ there is a branch B_i of T at i such that*

$$|\{q \in Q - X_i : T_i(q) = B_i\}| > \tfrac{1}{2}|Q - X_i|.$$

Let e_i be the edge of T incident with i and with some vertex of B_i. Now T has fewer edges than vertices, and so there exist distinct $j, j' \in I$ such that $e_j = e_{j'}$. Then j, j' are adjacent in T, and $j' \in V(B_j)$, $j \in V(B_{j'})$. Put $N = V(B_{j'})$, $N' = V(B_j)$. Then $N \cup N' = I$, and $N \cap N' = \emptyset$. Put $X_j \cap X_{j'} = A$, and put

$$W = \bigcup_{i \in N}(X_i - A), \qquad W' = \bigcup_{i \in N'}(X_i - A).$$

[p. 313] Then $W \cup W' = V(G) - A$ and $W \cap W' = \emptyset$. Without loss of generality we assume that

$$|W \cap Q| \geq |W' \cap Q|. \tag{1}$$

Now since $N' = V(B_j)$ and

$$\{q \in Q - X_j : T_j(q) = B_j\} = W' \cap Q,$$

we have $|W' \cap Q| > \frac{1}{2}|Q - X_j|$. But

$$Q - X_j = (W' \cap Q) \cup ((W - X_j) \cap Q)$$

and so

$$|(W - X_j) \cap Q| < |W' \cap Q|. \tag{2}$$

From (1) and (2) we deduce that there exists $Y \subseteq X_j - A$ such that

$$|(W - Y) \cap Q| = |W' \cap Q|.$$

Then $(W - Y, W')$ is a partition of $V(G) - (Y \cup A)$, and so

$$|(W - Y) \cap Q| = |W' \cap Q| = \tfrac{1}{2}|Q - (Y \cup A)|.^{F2}$$

But $W - Y$ and W' are separated in G by A, from (2.4), and so every component of $G \setminus (Y \cup A)$ has at most $\frac{1}{2}|Q - (Y \cup A)|$ vertices which are in Q. Also, $|Y \cup A| \leq w$, since $Y \cup A \subseteq X_j$. Thus we may take $X = Y \cup A$ and the theorem is satisfied.

(2.6) *If G has tree-width $< w$ and $Q \subseteq V(G)$, there exists a separation (H, H') of G such that $|V(H) \cap V(H')| \leq w$ and*

$$\begin{aligned} &|(V(H) - V(H')) \cap Q|, |(V(H') - V(H)) \cap Q| \\ &\quad \leq \tfrac{2}{3}|Q - (V(H) \cap V(H'))|. \end{aligned}$$

[A *separation* of G is a pair of subgraphs (H, H') of G, such that $V(H) \cup V(H') = V(G)$, $E(H) \cup E(H') = E(G)$, and $E(H) \cap E(H') = \emptyset$.]

Proof. It is sufficient to find a partition (X, Y, Y') of $V(G)$ such that $|X| \leq w$ and X separates Y, Y' in G and such that

$$|Y \cap Q|, |Y' \cap Q| \leq \tfrac{2}{3}|Q - X|.$$

[F2] **Misprint.** A misprint. The set Y was chosen with $|(W - Y) \cap Q| = |W' \cap Q|$, and $(W - Y) \cap Q$ and $W' \cap Q$ partition $Q - (Y \cup A)$, so each has $\frac{1}{2}|Q - (Y \cup A)|$ elements. As printed, the display is false whenever $Q \neq \varnothing$, since $|W' \cup Q| \geq |Q|$.

For then H, H' can be constructed with $V(H) = X \cup Y$, $V(H') = X \cup Y'$, and with the edges of G divided between H and H'. But since G has a [p. 314] tree-decomposition with width $< w$, there exists $X \subseteq V(G)$ with $|X| \leq w$ such that every component of $G \setminus X$ has at most $\frac{1}{2}|Q - X|$ vertices which are in Q, by (2.5).

Let $G \setminus X$ have components $C_1, C_2, \ldots, C_r$, where

$$|V(C_i) \cap Q| = c_i \qquad (1 \leq i \leq r)$$

and $c_1 \geq c_2 \geq \cdots \geq c_r$. Then $c_1 + \cdots + c_r = |Q - X|$, and so $c_1 \leq \frac{1}{2}(c_1 + \cdots + c_r)$, and $r \geq 2$. If $c_1 \geq \frac{1}{3}(c_1 + \cdots + c_r)$ we may take $Y = V(C_1)$, $Y' = V(C_2) \cup \cdots \cup V(C_r)$. We assume then that $c_1 < \frac{1}{3}(c_1 + \cdots + c_r)$. Choose i minimum such that

$$c_1 + \cdots + c_i \geq \tfrac{1}{3}(c_1 + \cdots + c_r).$$

Then

$$c_1 + \cdots + c_{i-1} \leq \tfrac{1}{3}(c_1 + \cdots + c_r),$$

and $c_i \leq c_1 \leq \frac{1}{3}(c_1 + \cdots + c_r)$, and so

$$c_1 + \cdots + c_i \leq \tfrac{2}{3}(c_1 + \cdots + c_r).$$

Thus we may take $Y = V(C_1) \cup \cdots \cup V(C_i)$, $Y' = V(C_{i+1}) \cup \cdots \cup V(C_r)$, and the theorem is satisfied.

For the algorithms of the next section, the only facts we need about tree-width are (2.6) and the following elementary observation.

(2.7) *If G has tree-width $< w$, so does every minor of G.*

We give the details. Let $(X_i : i \in I)$, T be a tree-decomposition of G with $|X_i| \leq w$ for all $i \in I$, and let H be a minor of G. Then there are disjoint non-empty sets $Y(v) \subseteq V(G)$ $(v \in V(H))$, each inducing a connected subgraph of G, and for each edge e of H an edge $f(e)$ of G, such that if e has ends u, v, then $f(e)$ has one end in $Y(u)$ and the other in $Y(v)$. (Take for $Y(v)$ the set of vertices of G that are contracted into v, and for $f(e)$ the edge of G that becomes e.) For $i \in I$ let X_i' be the set of all $v \in V(H)$ with $Y(v) \cap X_i \neq \emptyset$. Choosing for each $v \in X_i'$ a vertex in $Y(v) \cap X_i$ gives distinct vertices for distinct v, since the sets $Y(v)$ are disjoint, and so $|X_i'| \leq |X_i| \leq w$. We claim that $(X_i' : i \in I)$, T is a tree-decomposition of H, and this proves (2.7). For (W1), each $Y(v)$ is non-empty, and every vertex of G is in some X_i. For (W2), let e be an edge of H with ends u, v. Then $f(e)$ has ends $a \in Y(u)$ and $b \in Y(v)$, and $a, b \in X_i$ for some i, by (W2) for G. Then $u, v \in X_i'$. For

(W3), let j lie on the path of T from i to k, and let $v \in X'_i \cap X'_k$. Suppose that $Y(v) \cap X_j = \emptyset$. We claim that for all $a, b \in Y(v)$ and all $i', k' \in I$ with $a \in X_{i'}$ and $b \in X_{k'}$, there is a walk of T from i' to k' that does not pass through j. If $a = b$, then j is not on the path of T from i' to k', since otherwise $a \in X_{i'} \cap X_{k'} \subseteq X_j$ by (W3) for G, and we take this path. Since $Y(v)$ induces a connected subgraph of G, it is enough now to show that if the claim holds for a, b, and $c \in Y(v)$ is adjacent to b, then it holds for a, c. Let $a \in X_{i'}$ and $c \in X_{k'}$. By (W2) for G, some X_t contains b and c. There is a walk from i' to t not through j, since the claim holds for a, b, and a walk from t to k' not through j, by the case $a = b$ applied to c. Together they give a walk from i' to k' not through j. This proves the claim. Now choose $a \in Y(v) \cap X_i$ and $b \in Y(v) \cap X_k$. By the claim there is a walk of T from i to k not through j. But every walk of the tree T from i to k passes through every vertex of the path of T from i to k, and in particular through j. This is a contradiction, and so $Y(v) \cap X_j \neq \emptyset$, that is, $v \in X'_j$.[F3]

Our algorithm (1.3) to test feasibility of a terminated graph G takes advantage of the separation given by (2.6) with $Q = V(G)$ to reduce to two smaller problems, each with at most two-thirds (approximately) of the original vertices. There are basically two difficulties.

(i) This reduction can sometimes increase the index of the terminated graph, if the support of the original termination is not divided fairly evenly between the two sides of the separation, because the "separating" vertices are included in the supports of the new terminations. This is easily overcome; we simply apply (2.6) to both these smaller problems, this time with Q the support of the current termination, and reduce to four problems, each with index at most that of the original and each containing at most two-thirds of the original vertices.

(ii) The original terminated graph may be feasible only by means of paths which cross from H to H' and back several times, and our method of reduction to smaller problems must take account of this. For that reason we need the following technical device.

[p. 315] Let (H, H') be a separation of G, and let $\mathscr{A}_0 = \{(s_i, t_i) : 1 \leq i \leq k\}$ be a termination of G. Put $V(H) \cap V(H') = X$, $V(H) - V(H') = Y$, $V(H') - V(H) = Y'$. A *crossing scheme* with respect to (H, H') is a quadruple $(\mathscr{A}, \mathscr{A}', Z, Z')$, where $\mathscr{A}, \mathscr{A}'$ are terminations of G and $Z, Z' \subseteq X$, such that

(i) $S(\mathscr{A}) - X = S(\mathscr{A}_0) \cap Y$ and $S(\mathscr{A}') - X = S(\mathscr{A}_0) \cap Y'$,

[F3] **Missing proof.** The paper calls (2.7) an elementary observation and gives no proof. The argument above is the one used in the formalization.

(ii) every vertex of X is in an even number of $S(\mathscr{A}_0), S(\mathscr{A}), S(\mathscr{A}')$,

(iii) $Z \cap Z' = \emptyset$, and $Z \cup Z' = X - (S(\mathscr{A}_0) \cup S(\mathscr{A}) \cup S(\mathscr{A}'))$,

(iv) if F is the graph with vertex set $S(\mathscr{A}) \cup S(\mathscr{A}')$ and edge set $\mathscr{A} \cup \mathscr{A}'$, then F has precisely k components $F_1, \ldots, F_k$ say, and for $1 \le i \le k$, F_i is a path joining s_i and t_i.

(2.8) *The number of different crossing schemes for $\mathscr{A}_0$ with respect to (H, H') is at most*

$$\max(2, (|X| + k)^{|X|}).$$

The proof is routine~~, and is left to the reader~~. (This is only a crude bound, which we have used for simplicity.) All that we shall need from (2.8) is that the number of crossing schemes is bounded by a function depending only on $|X|$ and k, and not on the size of G.

We give the details. As in (iv), where they are the edges of F, we regard the members of a termination as unordered pairs. Let $n = |X|$, and let $(\mathscr{A}, \mathscr{A}', Z, Z')$ be a crossing scheme. By (i), $S(\mathscr{A}) \subseteq X \cup Y$ and $S(\mathscr{A}') \subseteq X \cup Y'$. So every vertex in $S(\mathscr{A}) \cap S(\mathscr{A}')$ is in X, and it is not in $S(\mathscr{A}_0)$, by (ii). Every vertex v of F is in $S(\mathscr{A}_0)$ or in $S(\mathscr{A}) \cap S(\mathscr{A}')$. For if v is in just one of $S(\mathscr{A}), S(\mathscr{A}')$, then $v \in S(\mathscr{A}_0)$, by (i) if $v \notin X$ and by (ii) if $v \in X$. Also every vertex of $S(\mathscr{A}_0)$ is in exactly one of $S(\mathscr{A}), S(\mathscr{A}')$, by (i) and (ii). Let $1 \le i \le k$. By (iv), the only vertices of $S(\mathscr{A}_0)$ in F_i are s_i and t_i, since the other vertices of $S(\mathscr{A}_0)$ are in other components of F. So every internal vertex of F_i is in $S(\mathscr{A}) \cap S(\mathscr{A}')$, and therefore in $X - S(\mathscr{A}_0)$. Let L_i be the sequence of internal vertices of F_i, in order from s_i to t_i. These are distinct vertices of $X - S(\mathscr{A}_0)$, and no vertex is in two of $L_1, \ldots, L_k$. Let M be the set of vertices of X that are in Z, or in $S(\mathscr{A}_0) \cap S(\mathscr{A})$. No vertex of M is in any L_i, since the vertices of the sequences L_i are in $S(\mathscr{A})$ and not in $S(\mathscr{A}_0)$, while Z is disjoint from $S(\mathscr{A})$ by (iii).

We claim that $L_1, \ldots, L_k$ and M determine the crossing scheme. First, F_i is the path s_i, L_i, t_i for $1 \le i \le k$, and every edge of F is in some F_i. So F is determined. Second, the vertices in the sequences L_i are exactly the vertices of $S(\mathscr{A}) \cap S(\mathscr{A}')$. For each vertex of $S(\mathscr{A}) \cap S(\mathscr{A}')$ is a vertex of F not in $S(\mathscr{A}_0)$, and so it is an internal vertex of some F_i. Third, $S(\mathscr{A})$ consists of these vertices together with the vertices of $S(\mathscr{A}_0)$ in $S(\mathscr{A})$, since every vertex of F is in $S(\mathscr{A}_0)$ or in $S(\mathscr{A}) \cap S(\mathscr{A}')$. A vertex of $S(\mathscr{A}_0)$ not in X is in $S(\mathscr{A})$ exactly when it is in Y, by (i), and one in X is in $S(\mathscr{A})$ exactly when it is in M. So $S(\mathscr{A})$ is determined. Fourth, $S(\mathscr{A}')$ consists of the vertices in the sequences L_i together with the vertices of $S(\mathscr{A}_0)$ not in $S(\mathscr{A})$, since each vertex of $S(\mathscr{A}_0)$ is in exactly one of $S(\mathscr{A}), S(\mathscr{A}')$. Fifth, $Z = M - S(\mathscr{A}_0)$, and then Z' is determined by (iii). Finally, $\mathscr{A}$ is determined

by F and $S(\mathscr{A})$. For let $\mathscr{B}$ be another set of disjoint pairs that are edges of F, with union $S(\mathscr{A})$. Each vertex of $S(\mathscr{A})$ is in one member of $\mathscr{A}$ and one member of $\mathscr{B}$, and so every vertex has valency 0 or 2 in the graph with edge set $(\mathscr{A} - \mathscr{B}) \cup (\mathscr{B} - \mathscr{A})$. If this graph had an edge it would have a circuit, which is impossible since it is a subgraph of the forest F. So $\mathscr{B} = \mathscr{A}$. In the same way $\mathscr{A}'$ is determined by F and $S(\mathscr{A}')$. This proves the claim.

Call an *arrangement* on a set W any k sequences $L_1, \ldots, L_k$ of distinct vertices of W, no vertex in two of them, together with a set M of vertices of W in none of them. Let $w = |W|$. We show by induction on w that there are at most $\prod_{1 \le j \le w}(k + j + 1)$ arrangements on W. For $w = 0$ there is one. Let $w > 0$ and $x \in W$. Removing x from an arrangement on W gives an arrangement on $W - \{x\}$, and the arrangement on W is determined by this and one of the following: $x \in M$; x is in no sequence and not in M; x is first in L_i, for some i; x comes just after y in some sequence, for some $y \in W - \{x\}$. These are $2 + k + w - 1 = k + w + 1$ possibilities, as required. Let $P_w = \prod_{1 \le j \le w}(k + j + 1)$. Then $P_w \le (k + w)^w$ for $w \ge 3$, by induction on w. For $w = 3$ this holds since $(k + 2)(k + 4) \le (k + 3)^2$. If it holds for some $w \ge 3$, then since $(k + w + 2)(k + w) \le (k + w + 1)^2$,

$$\begin{aligned} P_{w+1} = (k + w + 2)P_w &\le (k + w + 2)(k + w) \cdot (k + w)^{w-1} \\ &\le (k + w + 1)^2(k + w + 1)^{w-1} = (k + w + 1)^{w+1}. \end{aligned}$$

By the claim, the number of crossing schemes is at most the number of arrangements on X, which is at most $P_n \le (n + k)^n$ if $n \ge 3$. This proves (2.8) when $n \ge 3$.

Now let $n \le 2$, and suppose first that some $a \in X$ is in $S(\mathscr{A}_0)$. Then a is in no sequence L_i, and so the arrangement given by a crossing scheme is determined by the arrangement on $X - \{a\}$ obtained by removing a, and by whether $a \in M$. There are at most P_{n-1} arrangements on $X - \{a\}$, which is 1 if $n = 1$ and $k + 2$ if $n = 2$. So there are at most 2 crossing schemes if $n = 1$, and at most $2(k + 2) \le (k + 2)^2$ if $n = 2$.

Next suppose that no vertex of X is in $S(\mathscr{A}_0)$. If $n = 0$, there is only one arrangement on X, and so at most one crossing scheme. Let $n \ge 1$. For $1 \le i \le k$, every member of $\mathscr{A}$ that meets F_i is an edge of F_i, and so $|S(\mathscr{A}) \cap V(F_i)|$ is even. This set consists of the vertices of L_i and those of s_i, t_i that are in $S(\mathscr{A})$. Since $s_i, t_i \notin X$, by (i) these are the ones in Y. So $|L_i| + |\{s_i, t_i\} \cap Y|$ is even, and the parity of $|L_i|$ is the same for all crossing schemes. Suppose that $|L_i|$ is odd for some i. Then $|L_i| = 1$, since $n \le 2$, and there are at most n choices for its vertex x. At most one other vertex y of X remains, and for $j \ne i$, L_j is the sequence y if $|L_j|$ is odd, and is

empty otherwise. So the sequences are determined by x, and $M \subseteq \{y\}$ gives at most two choices for M. This gives at most $2n$ crossing schemes, and $2n \leq 2$ if $n = 1$, while $2n = 4 \leq (k+2)^2$ if $n = 2$. Suppose finally that every $|L_i|$ is even. If $n = 1$, all L_i are empty and there are two choices for M. If $n = 2$, either all L_i are empty and there are four choices for M, or one L_i consists of both vertices of X and $M = \emptyset$, which gives $2k$ choices. So there are at most $4 + 2k \leq (k+2)^2$ crossing schemes. This proves (2.8).[F4]

Now let $(\mathscr{A}, \mathscr{A}', Z, Z')$ be a crossing scheme for $\mathscr{A}_0$ with respect to (H, H'). Let $\bar{Z} = Z \cup ((S(\mathscr{A}_0) \cap X) - S(\mathscr{A}))$ and $\bar{Z}' = Z' \cup ((S(\mathscr{A}_0) \cap X) - S(\mathscr{A}'))$. Then $\mathscr{A}$ is a termination for $H \setminus \bar{Z}\bar{Z}$ and $\mathscr{A}'$ is a termination for $H' \setminus \bar{Z}'\bar{Z}'$. We call $(H \setminus \bar{Z}\bar{Z}, \mathscr{A})$, $(H' \setminus \bar{Z}'\bar{Z}', \mathscr{A}')$ the terminated graphs *induced* by the crossing scheme.[F5]

(2.9) *Let $(G, \mathscr{A}_0)$ be a terminated graph and let (H, H') be a separation of G. Then $(G, \mathscr{A}_0)$ is feasible if and only if there is a crossing scheme for $\mathscr{A}_0$ with respect to (H, H') such that both induced terminated graphs are feasible.*

This is clear once the various definitions have been digested~~, and it is left to the reader~~. The purpose of Z and Z' is to allow for the possibility that the desired paths may need to use the "separating" vertices without passing from one side of the separation to the other.

We give the details, with $\bar{Z}, \bar{Z}'$ as above. Suppose first that $(G, \mathscr{A}_0)$ is feasible, and let $P_1, \ldots, P_k$ be as in the definition of feasibility. Since $E(H) \cap E(H') = \emptyset$, each edge of P_i is in exactly one of H, H'. Divide P_i into maximal subpaths whose edges are all in H or all in H', and call them the *pieces* of P_i, of the first or second kind. Each piece has at least one edge, since $s_i \neq t_i$. Two consecutive pieces of P_i are of different kinds, and they share one vertex, which is in X, since it is incident with an edge of H and an edge of H'. Let $\mathscr{A}$ be the set of pairs of ends of the pieces of the

[F4] **Missing proof.** The paper leaves the proof to the reader. The argument above is the one used in the formalization. For $|X| \leq 2$ the product bound alone is too weak, and the parity of the paths is needed.

[F5] **Statement mistake.** With the printed definition, (2.9) is false, because a path of an induced graph may pass through a vertex of X that is a terminal of $\mathscr{A}_0$ given to the other side. Take the star G with center 0 and leaves $1, 2, 3$, with $\mathscr{A}_0 = \{(2,3), (0,1)\}$, and let H, H' have all four vertices, with edges $02, 03$ in H and 01 in H'. Then $X = V(G)$, and $\mathscr{A} = \{(2,3)\}$, $\mathscr{A}' = \{(0,1)\}$, $Z = Z' = \emptyset$ is a crossing scheme. Both induced graphs are feasible, by the path $2, 0, 3$ in H and the edge 01 in H'. But $(G, \mathscr{A}_0)$ is not feasible, since every path from 2 to 3 uses the terminal 0. The corrected definition also deletes from each side the terminals of $\mathscr{A}_0$ in X that the side's own termination does not use. Such a vertex is not in $S(\mathscr{A})$ on the side of H, and not in $S(\mathscr{A}')$ on the side of H', so $\mathscr{A}$ and $\mathscr{A}'$ are still terminations of the smaller graphs, and the bound of (2.10) is unchanged. With this definition (2.9) holds.

first kind, and $\mathscr{A}'$ the same for the second kind. Two pieces of the same kind are disjoint, and so $\mathscr{A}$ and $\mathscr{A}'$ are terminations. An internal vertex of a piece is not an end of any piece, and not in $S(\mathscr{A}_0)$. Let Z' be the set of vertices of X that are internal vertices of pieces of the first kind, and let Z be the set of vertices of $X - (S(\mathscr{A}_0) \cup S(\mathscr{A}) \cup S(\mathscr{A}'))$ not in Z'. We check (i)–(iv). For (i), an end of a piece of the first kind is a vertex of H, and it is either in X or an end of some P_i. Conversely, if $s_i \in Y$, then the edge of P_i at s_i is not in H', since $s_i \notin V(H')$, and so $s_i \in S(\mathscr{A})$. The same holds for t_i, and similarly for Y'. For (ii), let $x \in X$. If x is on no P_i, or is an internal vertex of a piece, it is in none of the three supports. If x is s_i or t_i, it is an end of exactly one piece. Otherwise x is the common end of two consecutive pieces, of different kinds. In each case x is in two or none of $S(\mathscr{A}_0), S(\mathscr{A}), S(\mathscr{A}')$. Condition (iii) holds by the definition of Z and Z'. For (iv), the ends of the pieces of P_i, in order along P_i, form a path F_i of F from s_i to t_i, and the paths $F_1, \dots, F_k$ are disjoint and contain all vertices and edges of F. Finally, each piece of the first kind is a path of H that does not meet $\bar{Z}$. For its ends are in $S(\mathscr{A})$, and its internal vertices are not in $S(\mathscr{A}_0)$, and are in Y or in Z'. Since these pieces are disjoint, $(H \setminus \bar{Z}, \mathscr{A})$ is feasible, and similarly $(H' \setminus \bar{Z}', \mathscr{A}')$ is feasible.

Conversely, let $(\mathscr{A}, \mathscr{A}', Z, Z')$ be a crossing scheme such that both induced terminated graphs are feasible. Then for each pair $p \in \mathscr{A}$ there is a path Q_p of $H \setminus \bar{Z}$ joining the two vertices of p, and for each $p \in \mathscr{A}'$ a path Q_p of $H' \setminus \bar{Z}'$, such that paths for different pairs of the same termination are disjoint. (As in the proof of (2.8), no pair is in both $\mathscr{A}$ and $\mathscr{A}'$.) We claim that if $p \in \mathscr{A}$ and $q \in \mathscr{A}'$, then Q_p and Q_q meet only in common vertices of p and q. For let v be a vertex of both. Then $v \in X$, and $v \notin \bar{Z} \cup \bar{Z}'$. If $v \in S(\mathscr{A}_0)$, then $v \in S(\mathscr{A}) \cap S(\mathscr{A}')$ by the definition of $\bar{Z}, \bar{Z}'$, contrary to (ii). So $v \notin S(\mathscr{A}_0)$. Since $v \notin Z \cup Z'$, it follows from (iii) that v is in $S(\mathscr{A})$ or $S(\mathscr{A}')$, and then from (ii) that it is in both. Hence v is a vertex of some $p' \in \mathscr{A}$, and so $v \in Q_{p'}$, and $p' = p$ since the paths Q_r ($r \in \mathscr{A}$) are disjoint. Similarly v is a vertex of q, as claimed. Now for $1 \le i \le k$, let P_i be the union of the paths Q_p for the edges p of F_i. Two consecutive edges of F_i are in different terminations, since the pairs of a termination are disjoint, and so their paths meet only in their common vertex. Two edges of F that are not consecutive edges of the same F_i have no common vertex, and so their paths are disjoint. Hence P_i is a path of G joining s_i and t_i, and $P_1, \dots, P_k$ are pairwise disjoint. Thus $(G, \mathscr{A}_0)$ is feasible.[F6]

[F6] **Missing proof.** The paper leaves the proof to the reader. The argument above, for the corrected definition of the induced terminated graphs, is the one used in the formalization.

We also have

(2.10) *The index of any terminated graph induced by a crossing scheme for $\mathscr{A}_0$ with respect to (H, H') is at most*

$$\tfrac{1}{2}|V(H) \cap V(H')| + \tfrac{1}{2}\max(|(V(H) - V(H')) \cap S(\mathscr{A}_0)|, \\ |(V(H') - V(H)) \cap S(\mathscr{A}_0)|).$$

[p. 316] 3. The Basic Algorithm

Now we give the details of our main algorithm. We require first

(3.1) *For any fixed integer w, there is a polynomial algorithm which has input a simple graph G and a subset Q of $V(G)$, and which has output either*

(i) *a separation (H, H') of G with $|V(H) \cap V(H')| \le w$, such that*

$$|(V(H) - V(H')) \cap Q|, |(V(H') - V(H)) \cap Q| \\ \le \tfrac{2}{3}|Q - (V(H) \cap V(H'))|,$$

or

(ii) *a (valid) statement that no such separation exists.*

Proof. We examine all subsets $X \subseteq V(G)$ of cardinality $\le w$, and check if some component of $G \setminus X$ has more than $\frac{2}{3}|Q - X|$ vertices which are in Q. If this is true for all such X, then no separation of the required kind exists; and if not we can produce H, H', as in the proof of (2.6). Clearly this is a polynomial algorithm, since w is fixed. Moreover, the running time is bounded by a polynomial in $|V(G)|$, since G is simple. This concludes the proof.

We shall inductively define two algorithms $A_1(n)$ and $A_2(n)$, for $n \ge 1$. Throughout, w is a fixed integer. $A_1(n)$ has as input a terminated graph $(G, \mathscr{A})$ with index $\le \frac{5}{2}w$, where G is simple and has at most n vertices. It either decides whether $(G, \mathscr{A})$ is feasible or outputs that G has tree-width $\ge w$. $A_2(n)$ has as input a terminated graph $(G, \mathscr{A})$ with index $\le 3w$, where G is simple and has at most n vertices. It either decides whether $(G, \mathscr{A})$ is feasible or outputs that G has tree-width $\ge w$.

Definition of $A_1(n)$. If $n \le 4w$ proceed by exhaustion. If $n > 4w$ (this restriction is only of importance later when analyzing running time) take two steps.

Step 1. Use the algorithm of (3.1) with $Q = V(G)$ to find a separation (H, H') of G where $|V(H) \cap V(H')| \le w$ and

$$|V(H) - V(H')|, |V(H') - V(H)| \le \tfrac{2}{3}|V(G) - (V(H) \cap V(H'))|.$$

If no such separation exists, announce (by (2.6)) that G has tree-width $\geq w$, and stop. If the separation has been found, go to step 2.

Step 2. Generate all crossing schemes for $\mathscr{A}$ with respect to (H, H'), and for each apply $A_2(\frac{2}{3}n + \frac{1}{3}w)$ to the two induced terminated graphs. If in one of these applications it is found that the graph has tree-width $\geq w$, announce that G has tree-width $\geq w$. If not, then the feasibility of all these [p. 317] induced terminated graphs has been found. Then decide if $(G, \mathscr{A})$ is feasible using (2.9).

We observe that $A_2(\frac{2}{3}n + \frac{1}{3}w)$ is applicable here, since

$$|V(H)|, |V(H')| \leq \tfrac{2}{3}n + \tfrac{1}{3}w$$

and the induced terminated graphs all have index $\leq \frac{5}{2}w + \frac{1}{2}w = 3w$, by (2.10).

Definition of $A_2(n)$. This is identical with the definition of $A_1(n)$, except that in step 1 we use (3.1) with $Q = S(\mathscr{A})$ instead of with $Q = V(G)$, and in step 2 we apply $A_1(n)$ instead of $A_2(\frac{2}{3}n + \frac{1}{3}w)$.

We observe that $A_1(n)$ is applicable here, since

$$|V(H)|, |V(H')| \leq n$$

and, by (3.1) and (2.10), the induced terminated graphs all have index at most

$$\tfrac{1}{2}|V(H) \cap V(H')| + \tfrac{1}{2} \cdot \tfrac{2}{3} \cdot 6w \leq \tfrac{5}{2}w.$$

This completes the definition of $A_1(n)$ and $A_2(n)$.

Let $f_1(n)$ and $f_2(n)$ denote the respective worst-case running times of $A_1(n)$ and $A_2(n)$. Let the algorithm of (3.1) have running time $\leq c_1 n^{d_1}$ for $m \geq 1$, when $|V(G)| = n$. Then from (2.8) we have

$$\begin{aligned}
\text{if } n > 4w \quad \text{then } f_1(n) &\leq c_1 n^{d_1} + c_2 + c_3 f_2(\tfrac{2}{3}n + \tfrac{1}{3}w) \\
&\leq c_1 n^{d_1} + c_2 + c_3 f_2(\tfrac{3}{4}n) \\
&\leq c(n^{d_1} + f_2(\tfrac{3}{4}n)), \\
\text{if } n > 4w \quad \text{then } f_2(n) &\leq c_1 n^{d_1} + c_4 + c_5 f_1(n) \\
&\leq c(n^{d_1} + f_1(n)), \\
\text{if } n \leq 4w \quad \text{then } f_1(n), f_2(n) &\leq c,
\end{aligned}$$

where $c_2, \ldots, c_5, c$ are appropriate constants depending only on w. Then for $n > 4w$,

$$f_2(n) \leq (c + c^2)n^{d_1} + c^2 f_2(\tfrac{3}{4}n)$$

and so $f_2(n)$ is bounded by a polynomial in n. The algorithms $A_2(n)$ $(n > 0)$ therefore provide a polynomial algorithm A_2 (say), which, given as input a terminated graph $(G, \mathscr{A})$ of index $\leq 3w$ where G is simple, either decides if $(G, \mathscr{A})$ is feasible or announces that G has tree-width $\geq w$.

[p. 318] It follows that

(3.2) *For any two fixed integers w, k there is a polynomial algorithm which has input a terminated graph $(G, \mathscr{A})$ of index $\leq k$, and outputs either*

(i) *$(G, \mathscr{A})$ is feasible, or*

(ii) *$(G, \mathscr{A})$ is not feasible, or*

(iii) *G has tree-width $> w$.*

Proof. Define $w' = 1 + \max(w, k/3)$, and apply algorithm A_2, with w' replacing w, to a pair $(G', \mathscr{A})$ where G' is the underlying simple graph of G.

4. Applications of the Basic Algorithm

In this section we derive (1.1), which is the main result of this paper, and we restate it for the reader's convenience.

(4.1) *For any fixed planar graph H, there is a polynomial algorithm to decide if the input graph has a minor isomorphic to H.*

Let v be a vertex of a graph G which is incident with only two edges e_1, e_2, and suppose e_1, e_2 are not loops. Let e_i have ends v, v_i $(i = 1, 2)$, so that $v_1, v_2 \neq v$. The result of *suppressing* v is the graph obtained from G by deleting v and adding a new edge with ends v_1, v_2.

We say that G *topologically contains* H if H can be obtained from a subgraph of G by repeated suppression. We observe that if G topologically contains H then $V(H) \subseteq V(G)$. There is a connection between topological containment and minors, for clearly if G topologically contains H then G has a minor isomorphic to H. The converse does not hold, however—it is possible for G to have a minor isomorphic to H and yet not to topologically contain any graph isomorphic to H.

The following two results are clear.

(4.2) *If $V(H) \subseteq V(G)$ and e is a loop of G incident with $v \in V(H)$, and no loop of H is incident with v, then G topologically contains H if and only if $G \setminus e$ does.*

(4.3) *If $V(H) \subseteq V(G)$, and e is a loop of G and f is a loop of H, both incident with the same vertex, then G topologically contains H if and only if $G \setminus e$ topologically contains $H \setminus f$.*

Now let $V(H) \subseteq V(G)$, and suppose that no loop of G is incident with a vertex of H. We construct a terminated graph $(G', \mathscr{A})$ as follows. Let the edges of H be $e_1, \ldots, e_k$, and let e_i have ends $u_i, v_i \in V(G)$ $(1 \le i \le k)$.

[p. 319] Take a set X of $2k$ new vertices $\{s_1, t_1, \ldots, s_k, t_k\}$. Let $Y = V(G) - V(H)$, and for $v \in X \cup Y$, define

$$\begin{aligned} g(v) &= v \text{ if } v \in Y \\ &= u_i \text{ if } v = s_i \ (1 \le i \le k) \\ &= v_i \text{ if } v = t_i \ (1 \le i \le k). \end{aligned}$$

Define a simple graph G' with a vertex set $X \cup Y$, in which distinct $a, b \in X \cup Y$ are adjacent if and only if $g(a), g(b)$ are adjacent in G. Let $\mathscr{A} = \{(s_i, t_i) : 1 \le i \le k\}$. Then $(G', \mathscr{A})$ is a terminated graph. We evidently have

(4.4) *G' has tree-width at most $2k$ more than the tree-width of G.*

This follows because we may obtain a subgraph of G by deleting $2k$ vertices of G'. Moreover, it is easy to see the following.

(4.5) *If H has no loops and no two edges with the same ends, then G topologically contains H if and only if $(G', \mathscr{A})$ is feasible.*[F7]

We use (4.2)–(4.5) to give an algorithm as follows.

(4.6) *For any fixed graph H_0 and fixed integer w, there is a polynomial algorithm which has input graphs G, H with $V(H) \subseteq V(G)$ such that H is isomorphic to a subgraph of H_0, and which has output either*

(i) *G topologically contains H, or*

(ii) *G does not topologically contain H, or*

(iii) *G has tree-width $> w$.*

Proof. If some vertex of H is incident with a loop of G, we use (4.2) or (4.3) to eliminate the loop. If there is no such vertex, we construct $(G', \mathscr{A})$ as for (4.4) and (4.5), and apply (3.2) to $(G', \mathscr{A})$ with w replaced by $w+2k$.[F8] We find either

[F7] **Statement mistake.** The printed (4.5) is false when H has a loop or two edges with the same ends, because G' forgets which edge of G joins two vertices, so two paths of G' may use the same edge of G. Take $V(G) = V(H) = \{u, v\}$, let G have one edge uv, and let H have two edges with ends u, v. Then s_1t_1 and s_2t_2 are disjoint edges of G', so $(G', \mathscr{A})$ is feasible, but G does not topologically contain H. For a loop, take $V(G) = \{u, x\}$ with one edge ux, and let H be the vertex u with one loop. The path s_1, x, t_1 shows feasibility, but G has no cycle. The direction from containment to feasibility holds for every H.

[F8] **Proof gap.** By (4.5) as corrected, this step is right only when H has no loops and no two edges with the same ends, and H, a subgraph of H_0, may have them. The proof is corrected by changing the construction of G' so that it remembers the edges of G. Put one new vertex on each edge of G, and join each new terminal s_i or t_i only to the vertex

(i) $(G', \mathscr{A})$ is feasible, or
(ii) $(G', \mathscr{A})$ is not feasible, or
(iii) G' has tree-width $> w + 2k$.

For the first two alternatives we use (4.5) to decide whether or not G topologically contains H, and for the third we deduce from (4.4) that G has tree-width $> w$.

(4.7) *For any fixed graph H and fixed integer w, there is a polynomial algorithm which has input a graph G, and which has output one of*
(i) *there is a graph isomorphic to H topologically contained by G, or*
(ii) *there is no such graph, or*
(iii) *G has tree-width $> w$.*

[p. 320] *Proof.* We use (4.6) to try the $n(n-1)\cdots(n-p+1)$ injections of $V(H)$ into $V(G)$, one by one (where $|V(G)| = n$ and $|V(H)| = p$).

(4.8) *For any graph H there is a finite set $H_1, \ldots, H_r$ of graphs such that for any graph G, G has a minor isomorphic to H if and only if G topologically contains a graph isomorphic to one of $H_1, \ldots, H_r$.*

This is elementary. (We "split" every vertex of H with valency ≥ 4 into two adjacent vertices of valency ≥ 3, repeatedly, in all possible ways.)

We give the details. Valencies count a loop twice. Let L be a graph with an edge f whose ends x, y are distinct and both have valency ≥ 3, and let K be obtained from L by contracting f. The new vertex of K has valency equal to the sum of the valencies of x and y minus 2, which is at least 4, and we say that L is obtained from K by *splitting* it. Call a graph a *split* of H if it can be obtained from a graph isomorphic to H by a sequence of such steps. Each step adds one vertex and one edge. It raises the sum of the valencies by 2, and the sum over all vertices v of $\min(d(v), 3)$ by 3, where $d(v)$ is the valency of v. Since the second sum is at most the first, a split of H arises after at most $2|E(H)|$ steps, and so it has at most $|V(H)| + 2|E(H)|$ vertices and at most $3|E(H)|$ edges. Hence there are only finitely many splits of H up to isomorphism. Let $H_1, \ldots, H_r$ contain one of each.

If G topologically contains a graph isomorphic to some H_i, then G has a minor isomorphic to H_i, as observed above, and H_i has a minor isomorphic to H, obtained by contracting edges. So G has a minor isomorphic to H.

---

of one chosen edge of G at u_i or v_i. The two ends of a loop of H choose different edges. Since k is fixed, there are polynomially many choices, and G topologically contains H if and only if the terminated graph is feasible for some choice. If G has tree-width at most w, this graph has tree-width at most $\max(w, 2) + 2k$, so (3.2) is applied with w replaced by $\max(w, 2) + 2k$, and alternative (iii) below changes in the same way. These two facts take the place of (4.5) and (4.4) in the rest of the proof.

Conversely, suppose that G has a minor isomorphic to H. Then G has a subgraph J with vertex-disjoint subtrees T_v ($v \in V(H)$) containing all vertices of J, such that the edges of J in no T_v correspond one-to-one to the edges of H, an edge of H with ends u, v corresponding to an edge of J with one end in $V(T_u)$ and the other in $V(T_v)$. (Take a subgraph of G contractible to a graph isomorphic to H, and keep a spanning tree of each set of vertices that is contracted to one vertex, together with the edges that are not contracted.) Call the edges of J in no T_v the *outer* edges, and their ends *marks*. We change J in three steps, keeping these properties, and keeping that G topologically contains J.

First, replace each T_v that contains a mark by the smallest subtree of T_v containing all marks in T_v, and each other T_v by one of its vertices, and delete all other vertices of J. Now every leaf of a tree T_v with at least two vertices is a mark, and so every vertex of such a tree has valency ≥ 2 in J.

Second, while some vertex x of a tree T_v with at least two vertices has valency 2, suppress x. This is possible, since x is incident with an edge of T_v, and so with no loop. If both edges at x are in T_v, the new edge joins two vertices of T_v, and we add it to $T_v - x$, which is then a tree. Otherwise x is a leaf of T_v, the edges at x are an edge of T_v with other end z and an outer edge e, and the new edge joins z to the other end of e. We let it correspond to the same edge of H as e, and replace T_v by $T_v - x$. In both cases the valencies of the other vertices do not change. Each suppression removes a vertex, and so this stops. Then every vertex of a tree T_v with at least two vertices has valency ≥ 3.

Third, we show by induction on the number of edges in the trees T_v that J is a split of H. If there are none, every T_v has one vertex, and J is isomorphic to H. Otherwise let f be an edge of some T_v, with ends x, y. Both have valency ≥ 3. Contracting f gives a graph with trees as above, with fewer edges in its trees, in which again every vertex of a tree with at least two vertices has valency ≥ 3, since the new vertex has valency ≥ 4. By induction it is a split of H, and so J is a split of H, since it is obtained from that graph by splitting the new vertex. Hence J is isomorphic to some H_i, and G topologically contains J, as required.[F9]

(4.9) *For any fixed graph H and fixed number w, there is a polynomial algorithm which has input a graph G, and has output either*

(i) *G has a minor isomorphic to H, or*

(ii) *G has no minor isomorphic to H, or*

[F9] **Missing proof.** The paper sketches this proof in one sentence. The argument above fills in the details, and it is the one used in the formalization.

(iii) *G has tree-width $> w$.*

Proof. We apply (4.7) to G, testing for each of the graphs $H_1, \ldots, H_r$ of (4.8) in turn.

Theorem (4.1) follows from (4.8) and the following.

(4.10) [12]. *For every planar graph H there is a number w such that every graph with no minor isomorphic to H has tree-width $\leq w$.*

The algorithm for (4.1) is as follows. Given a graph G, we apply the algorithm of (4.9) to it, with w as in (4.10). Each of the three possible outcomes determines whether or not G has a minor isomorphic to H.

5. An Algorithm for Tree-Width

Now we prove theorem (1.2). The following is shown in [11].

(5.1) *Let w be a number, and let $\mathscr{C}$ be a set of graphs each with tree-width $\leq w$. Suppose that no member of $\mathscr{C}$ is isomorphic to a minor of another member. Then $\mathscr{C}$ is finite.*

We deduce

(5.2) *For any fixed number w, there is a finite set of graphs $H_1, \ldots, H_r$ such that for any graph G, G has tree-width $> w$ if and only if G has a minor isomorphic to one of $H_1, \ldots, H_r$.*

Proof. Let $\mathscr{C}'$ be the set of all graphs H such that H has tree-width $> w$ but every minor of H distinct from H has tree-width $\leq w$. Then every [p. 321] member of $\mathscr{C}'$ has tree-width $w+1$, as is easily seen. By (2.7), a graph has tree-width $> w$ if and only if it has a minor in $\mathscr{C}'$. Choose a representative from each isomorphism class of $\mathscr{C}'$, and let the set of these representatives be $\mathscr{C}$. Now no member of $\mathscr{C}$ is isomorphic to a minor of another, and so $\mathscr{C}$ is finite by (5.1) (with $w+1$ replacing w). Moreover, a graph has tree-width $> w$ if and only if it has a minor isomorphic to a member of $\mathscr{C}$. This completes the proof.

(5.3) *For any fixed number w, there is a polynomial algorithm which, given a graph G, decides if G has tree-width $> w$.*

Proof. Let $H_1, \ldots, H_r$ be as in (5.2). We apply (4.9) to G with $H_1, \ldots, H_r$ in turn. If one of the applications yields that G has tree-width $> w$, we are done. Otherwise we decide whether or not G has a minor isomorphic to H_i, for $1 \leq i \leq r$, and hence decide whether or not G has tree-width $> w$, by (5.2).

We cannot claim to have constructed an algorithm here, because we do not know any way to find the required graphs $H_1, \ldots, H_r$. Thus this is merely a nonconstructive proof of the existence of the algorithm.

The argument for (5.3) can be extended to yield the following.

(5.4) *Let $\mathscr{F}$ be a set of graphs, such that for any two graphs G, G', if $G \in \mathscr{F}$ and G' is isomorphic to a minor of G then $G' \in \mathscr{F}$. Suppose that there is a number w such that all members of $\mathscr{F}$ have tree-width $\leq w$. Then there is a polynomial algorithm to test if an arbitrary graph is in $\mathscr{F}$.*

~~The proof is left to the reader.~~ We give the proof. We use the following result of [11], which is (1.5) there and which implies (5.1): for any number k, in any infinite sequence $G_1, G_2, \ldots$ of graphs each with tree-width $\leq k$, there exist $i < j$ such that G_i is isomorphic to a minor of G_j. Let $\mathscr{T}$ be the set of all graphs with tree-width $\leq w+1$ that are not in $\mathscr{F}$.

(1) *There is a finite set $\mathscr{C} \subseteq \mathscr{T}$ such that every member of $\mathscr{T}$ has a minor isomorphic to a member of $\mathscr{C}$.*

Suppose not. Then we may choose $H_1, H_2, \cdots \in \mathscr{T}$ in turn such that for each $j \geq 1$, H_j has no minor isomorphic to any of $H_1, \ldots, H_{j-1}$, since the set $\{H_1, \ldots, H_{j-1}\}$ does not satisfy (1). By the result above with $k = w+1$, there exist $i < j$ such that H_i is isomorphic to a minor of H_j, a contradiction. This proves (1).

Let $\mathscr{C} = \{H_1, \ldots, H_r\}$ be as in (1). We claim that a graph G is in $\mathscr{F}$ if and only if it has no minor isomorphic to one of $H_1, \ldots, H_r$. If $G \in \mathscr{F}$ and G has a minor isomorphic to H_i, then $H_i \in \mathscr{F}$ by the hypothesis on $\mathscr{F}$, which is impossible since $H_i \in \mathscr{T}$. Now let $G \notin \mathscr{F}$. Delete vertices of G one at a time, as long as the graph remains outside $\mathscr{F}$. This gives a subgraph M of G with $M \notin \mathscr{F}$, such that $M \setminus v \in \mathscr{F}$ for every $v \in V(M)$. Then M has tree-width $\leq w+1$. For this is true if M has no vertices. Otherwise let $v \in V(M)$. Then $M \setminus v$ has a tree-decomposition of width $\leq w$, since it is in $\mathscr{F}$, and adding v to every set of this tree-decomposition gives a tree-decomposition of M of width $\leq w+1$. Hence $M \in \mathscr{T}$, and by (1), M has a minor isomorphic to some H_i. Since M is a subgraph of G, so has G. This proves the claim.

The algorithm is now as in (5.3). We apply (4.9) to G with $H_1, \ldots, H_r$ in turn. If one of the applications yields that G has tree-width $> w$, then $G \notin \mathscr{F}$, since all members of $\mathscr{F}$ have tree-width $\leq w$. Otherwise we decide for $1 \leq i \leq r$ whether or not G has a minor isomorphic to H_i, and hence whether or not $G \in \mathscr{F}$. As for (5.3), this proves that the algorithm exists, but does not construct it.[F10]

Alternatively formulated, we have

(5.5) *Let $\mathscr{F}$ be any set of graphs, membership of which is preserved under*

[F10] **Missing proof.** The paper leaves this proof to the reader. The argument above, which extends the proof of (5.3), is the one used in the formalization.

isomorphism and under taking minors. Suppose that some planar graph is not in $\mathscr{F}$. Then there is a polynomial algorithm to test if an arbitrary graph is in $\mathscr{F}$.

This follows easily from (4.10) and (5.4). We conjecture that the second sentence can be removed from (5.5).

Independently of this paper, Arnborg, Corneil, and Proskurowski [1] have given a polynomial algorithm to test if a graph has tree-width $\leq k$, for fixed k, and have shown the problem of determining tree-width to be *NP*-complete. See also [4, 7] for related results.

Note added in proof. March 1986: We have recently proved both the conjectures of this paper. The general DISJOINT CONNECTING PATHS problem (for any fixed index) and the problem of deciding membership of an arbitrary minor-closed class (generalizing (5.5)) can both be solved by polynomial algorithms with running times $O(|V|^2 \cdot |E|)$. We have linear time algorithms for (3.2) and (4.1).

[p. 322] References

1. S. Arnborg, D. G. Corneil, and A. Proskurowski, Complexity of finding embeddings in a k-tree, preprint, 1984.
2. M. R. Garey and D. S. Johnson, "Computers and Intractability," Freeman, San Francisco, 1979.
3. R. M. Karp, On the complexity of combinatorial problems, *Networks* **5** (1975), 45–68.
4. L. M. Kirousis and C. H. Papadimitriou, Searching and pebbling, preprint, 1984.
5. J. B. Kruskal, Well-quasi-ordering, the tree theorem, and Vázsonyi's conjecture, *Trans. Amer. Math. Soc.* **95** (1960), 210–225.
6. J. F. Lynch, The equivalence of theorem proving and the interconnection problem, *ACM SIGDA Newsletter* **5** (3) (1975), 31–65.
7. N. Megiddo, S. L. Hakimi, M. R. Garey, D. S. Johnson and C. H. Papadimitriou, The complexity of searching a graph (preliminary version), *in* "Proc. 22nd IEEE Found. Of Comp. Sci. Symp.," 1981, pp. 376–381.
8. D. Richards, Complexity of single-layer routing, unpublished manuscript, 1981.
9. N. Robertson and P. D. Seymour, Graph minors. I. Excluding a forest, *J. Combin. Theory Ser. B* **35** (1983), 39–61.
10. N. Robertson and P. D. Seymour, Graph minors. III. Planar tree-width, *J. Combin. Theory Ser. B* **36** (1984), 49–64.

11. N. Robertson and P. D. Seymour, Graph minors. IV. Tree-width and well-quasi-ordering, submitted.
12. N. Robertson and P. D. Seymour, Graph minors. V. Excluding a planar graph, *J. Combin. Theory Ser. B*, to appear.
13. N. Robertson and P. D. Seymour, Graph minors. VII. Disjoint paths on a surface, submitted.
14. N. Robertson and P. D. Seymour, Graph minors. XIV. Excluding a nonplanar graph, in preparation.
15. P. D. Seymour, Disjoint paths in graphs, *Discrete Math.* **29** (1980), 293–309.
16. Y. Shiloach, A polynomial solution to the undirected two paths problem, *J. Assoc. Comput. Mach.* **27** (1980), 445–456.
17. C. Thomassen, 2-linked graphs, *European J. Combin.* **1** (1980), 371–378.